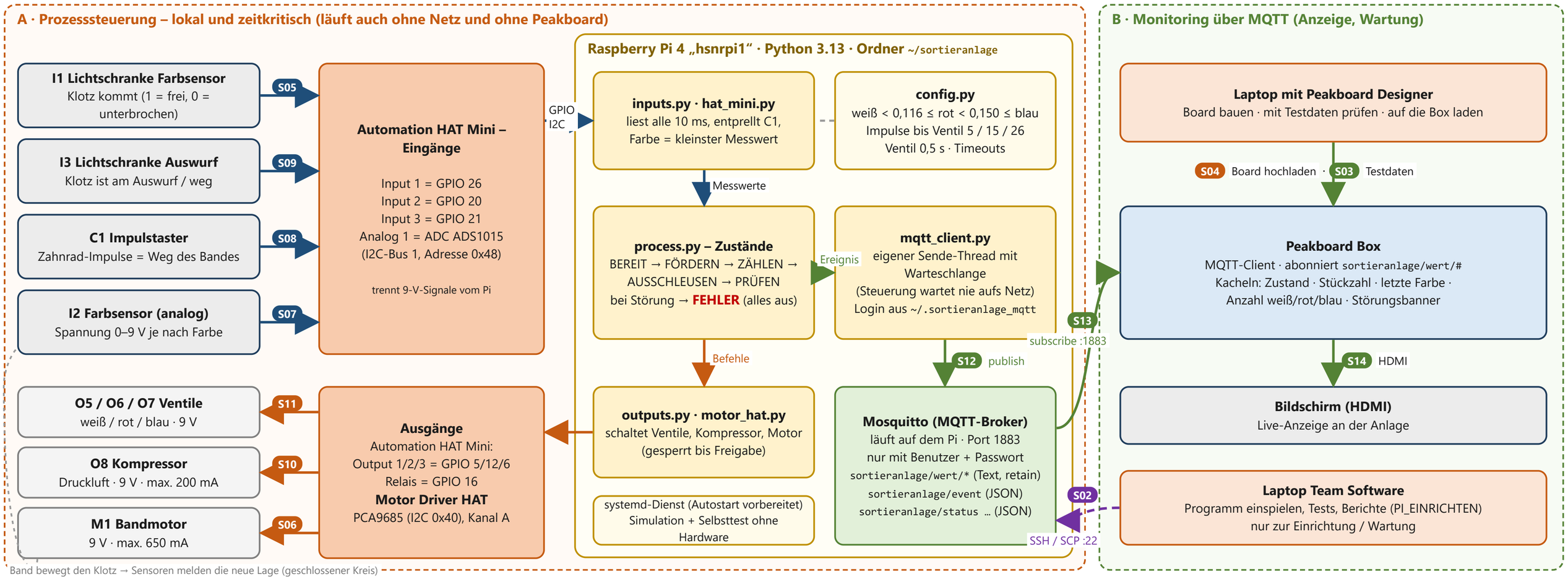
Datenflussplan V4 – Sortieranlage: Sensor → Automation HAT Mini → Raspberry Pi (Python) → Aktoren · MQTT → Peakboard (Stand 03.11.2026)



Jeder Pfeil = ein Datenfluss, in zeitlicher Reihenfolge (S01–S04 Inbetriebnahme · S05–S11 Weg eines Klotzes · S12–S14 Anzeige) · Status: BESTÄTIGT · VORSCHLAG · OFFEN

ID	von → nach	Inhalt	Signal / Format	Übertragung	Häufigkeit	Zweck	Status
S01	9-V-Netzteil → HATs + Aktoren · 5-V-Netzteil → Pi	Versorgung	9 V / 5 V 3 A	Kabel, WAGO-Klemmen	dauernd	Strom	9 V **vorhanden** · gemeinsame Masse **OFFEN**
S02	Laptop Team Software → Pi	Programm, Einrichtung, Tests, Berichte	SSH / SCP (Schlüssel)	WLAN · TCP 22	einmalig / Wartung	einspielen, warten	**BESTÄTIGT** (03.11.)
S03	Mosquitto → Peakboard Designer	Live-Testwerte (Simulation)	wie S13	WLAN · TCP 1883 · Login	beim Bauen	Board testen	**BESTÄTIGT** (Testdaten 03.11.)
S04	Peakboard Designer → Peakboard Box	fertiges Board	Peakboard-Projekt	WLAN/LAN · Port **OFFEN**	bei Änderung	Board hochladen	**OFFEN**
S05	I1 → HAT Input 1 (GPIO 26)	Klotz vor dem Farbsensor?	digital: 1 = frei, 0 = unterbrochen	Kabel · GPIO	Abfrage alle 10 ms	Messung starten	**BESTÄTIGT** (Team HW 20.10.)
S06	Pi → Motor Driver HAT (PCA9685, I2C 0x40, Kanal A) → M1	Band an / aus, Richtung	PWM Kanal 0, Richtung Kanal 1/2; 9 V, max. 650 mA	I2C-Bus 1 · Kabel	bei Zustandswechsel	Klotz transportieren	**BESTÄTIGT** · Schutz **OFFEN**
S07	I2 → HAT Analog 1 (ADS1015, I2C 0x48)	Farbwert	0–9 V → Zahl 0–1, kleinster Wert zählt	Kabel · I2C-Bus 1	alle 10 ms beim Messen	Farbe erkennen	**BESTÄTIGT** · Grenzen **kalibrieren**
S08	C1 → HAT Input 3 (GPIO 21)	Weg des Bandes	Impulse, entprellt (beide Flanken)	Kabel · GPIO	je Zahnradzahn	Klotz bis zum Ventil fahren	**BESTÄTIGT**
S09	I3 → HAT Input 2 (GPIO 20)	Klotz am Auswurf?	digital 1 / 0	Kabel · GPIO	alle 10 ms	Ausgang frei prüfen	**BESTÄTIGT**
S10	Pi → HAT Relais (GPIO 16) → Kompressor O8	Druckluft ein / aus	Relais, 9 V, max. 200 mA	Kabel	ab 1 s vor dem Ausschleusen	Druck aufbauen	**BESTÄTIGT**
S11	Pi → HAT Output 1–3 (GPIO 5/12/6) → Ventile O5–O7	Ventil auf / zu	digital, 9 V, Stoß 0,5 s	Kabel	1× je Klotz	aussortieren	**BESTÄTIGT** · Ventilstrom **OFFEN**
S12	mqtt_client.py → Mosquitto (localhost:1883)	Zustand, Stückzahl, letzte Farbe, Anzahl je Farbe, Fehler, Modus	MQTT: Text-Werte (retain) + JSON-Ereignis `{event, color, cycle_count, timestamp}`	TCP 1883 auf dem Pi, mit Login	bei jeder Änderung, je Klotz	Daten bereitstellen	**BESTÄTIGT** (Test 03.11.)
S13	Mosquitto → Peakboard Box	wie S12	Format A: `sortieranlage/wert/*` als Text	WLAN/LAN · TCP 1883 · Login	sofort bei Änderung	Anzeige	Format A **BESTÄTIGT** · Box-IP **OFFEN**
S14	Peakboard Box → Bildschirm	Dashboard	Bildsignal	HDMI	dauernd	anzeigen	**VORSCHLAG**

Beispiel 1 – Klotz rot wird richtig aussortiert: I1 wird unterbrochen (S05) → Band läuft (S06) → Farbsensor misst kleinsten Wert 0,13 (S07) → „rot“ → 15 Impulse zählen (S08) → Kompressor (S10), Ventil rot 0,5 s (S11) → I3 wieder frei (S09) → Stückzahl +1 → MQTT: `wert/farbe_letzte = rot`, `wert/anzahl_rot` +1, `event {"event":"sorted","color":"red"}` (S12) → Peakboard: rote Kachel, Zähler rot +1 (S13).

Beispiel 2 – Störung „Band blockiert“: Band läuft, aber 2 s lang kein Impuls von C1 (S08) → Zustand FEHLER → Motor, Ventile, Kompressor sofort AUS (S06, S10, S11) → MQTT: `wert/fehler_aktiv = 1`, `wert/fehler_text`, `event {"event":"error","code":"band_blockiert"}` (S12) → Peakboard: rotes Störungsbanner (S13). Fällt Peakboard oder das Netz aus, sortiert die Anlage trotzdem weiter.

Offen für die Abstimmung (M2): Farbwörter deutsch oder englisch (heute: Text-Werte deutsch, Ereignis englisch) · Topic-Anfang `sortieranlage/…` oder `hsnr/sortieranlage/…` · Ventilstrom, gemeinsame Masse, Schutz Motortreiber (Team Hardware) · Box-IP und Upload-Port (Team Netzwerk)